

The Stack at Work

Postfix Evaluation, Recursion & Tower of Hanoi

Unit I • Lecture 10 (final lecture of Unit I)

evaluate postfix • recursion in C • the call stack • Tower of Hanoi • tail recursion

14

5

9

Last time we made postfix — today we use it

Lecture 9 (conversion): $A + B * C - D \rightarrow A B C * + D -$

Lecture 10 (evaluation): $2\ 3\ 4\ * + 5 - \rightarrow 9$

- Same expression shape, but now the letters are numbers — so we can compute.
- Conversion stacked operators; evaluation stacks operands. Two stacks, two jobs.

The evaluation algorithm: two rules only

Algorithm: Infix to Postfix

1. Start with an empty stack and empty output.
2. Read the infix expression left to right, one symbol at a time.
3. If the symbol is an **operand** → add it to the output.
4. If the symbol is **(** → push it onto the stack.
5. If the symbol is an **operator** → pop to output while the stack top has **higher or equal** precedence (for **^**, only higher); then push the operator.
6. If the symbol is **)** → pop to output until **(** is found; discard both parentheses.
7. After the last symbol → pop all remaining operators to the output.
8. The output is the postfix expression. Stop.

Trace: 2 3 4 * + 5 - (infix: 2 + 3 × 4 − 5)

Symbol	Action	Stack after
2	number → push	2
3	number → push	2 3
4	number → push	2 3 4
*	pop 4, pop 3 → $3 \times 4 = 12$ → push	2 12
+	pop 12, pop 2 → $2 + 12 = 14$ → push	14
5	number → push	14 5
−	pop 5, pop 14 → $14 - 5 = 9$ → push	9
end	one number left — the answer	9

The simplest stack we can write

```
int stack[100];
int top = -1;

void push(int x)
{
    top = top + 1;
    stack[top] = x;
}

int pop(void)
{
    int x = stack[top];
    top = top - 1;
    return x;
}
```

- Deliberately bare: no struct, no guards — the Lecture 8 array stack stripped to its engine.
- Good enough here because our expressions are small and well-formed.
- In the lab version you should add the isEmpty and isFull checks from Lecture 8.

evaluatePostfix — the two rules in C

```
int evaluatePostfix(char expr[])
{
    for (int i = 0; expr[i] != '\0'; i++)
    {
        char ch = expr[i];

        if (ch == ' ')
            continue;          /* skip spaces */

        if (ch >= '0' && ch <= '9')
            push(ch - '0');    /* rule 1 */
        else
        {
            int right = pop();
            int left = pop();

            if (ch == '+') push(left + right);
            if (ch == '-') push(left - right);
            if (ch == '*') push(left * right);
            if (ch == '/') push(left / right);
        }
    }
    return pop();              /* the answer */
}
```

- One scan of the string; '\0' marks its end.
- `ch - '0'` turns the character '7' into the number 7.
- Operator branch is rule 2: pop two, apply, push one.
- The final pop returns the last number standing.

The one trap: which pop is which?

```
int right = pop();    /* popped FIRST */
int left  = pop();    /* popped SECOND */

push(left / right);
```

- The stack reverses order: the second operand is on top, so it pops out first.
- For + and * the mistake hides (they commute). Division and subtraction expose it.
- This is the most common lab bug in postfix evaluation — name the variables left and right and it disappears.

Try 6 2 /

push 6, push 2. On '/': first pop gives 2 (right), second gives 6 (left).

$\text{left} / \text{right} = 6 / 2 = 3$ ✓

Swap them and you compute $2 / 6 = 0$ ✗

Recursion: a function that calls itself

```
int factorial(int n)
{
    if (n == 0)
        return 1;          /* base case */

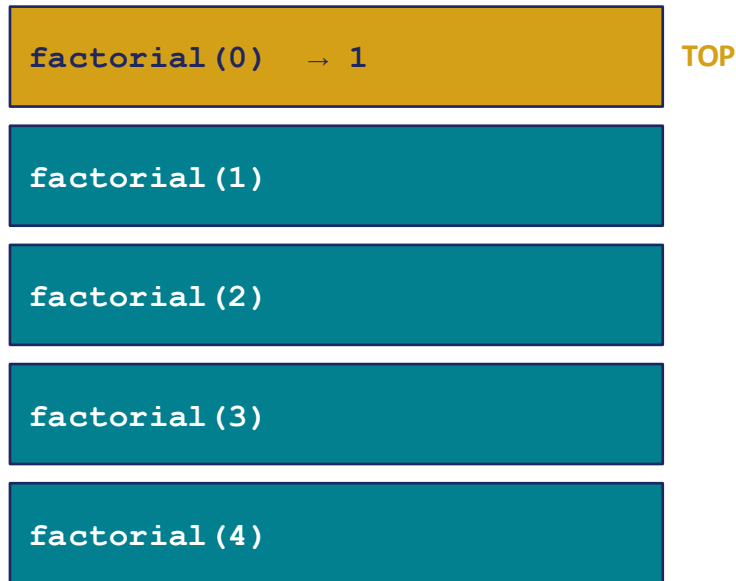
    return n * factorial(n - 1);
                          /* recursive case */
}
```

Read it as mathematics

$4! = 4 \times 3!$, $3! = 3 \times 2!$, ... $0! = 1$. The code is the definition, line for line.

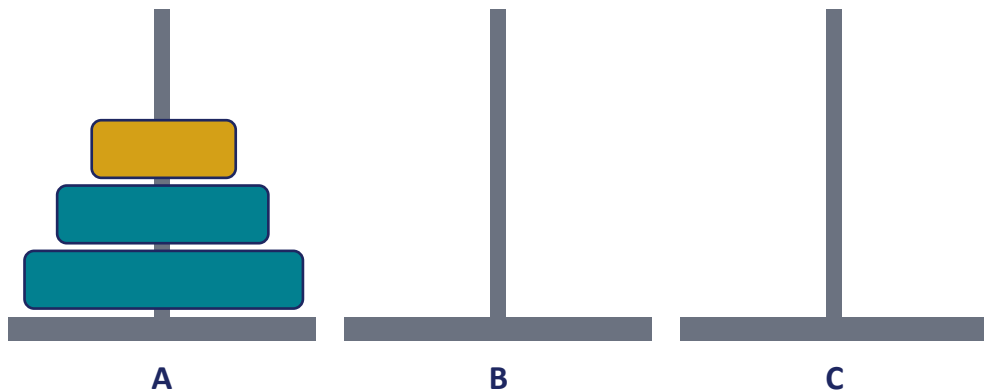
- Base case: the input so small the answer is known directly. Every recursion must have one.
- Recursive case: solve for n by using the answer for $n - 1$.
- No base case $\rightarrow$ the calls never stop $\rightarrow$ the program crashes (we will see exactly why next).

Where recursion lives: the call stack



- Every call PUSHES a frame (its n , its return address). Every return POPS one.
- `factorial(4)` stacks five frames, then they unwind: $1 \rightarrow 1 \rightarrow 2 \rightarrow 6 \rightarrow 24$.
- This is why recursion belongs in the stack unit — the machine runs it on a stack.
- No base case = pushes never stop = the stack fills up: stack overflow.

Tower of Hanoi: the problem



- Move all disks from peg A to peg C, using B as a helper.
- Rule 1: move one disk at a time — always the top disk of a peg.
- Rule 2: a larger disk may never sit on a smaller disk.
- Minimum moves for n disks: $2^n - 1$. For $n = 3$, that is 7 moves.

The recursive insight: three steps, whatever n is

```
ALGORITHM Hanoi(n, A, C, B)
// Moves n disks from peg A to peg C, using peg B as helper.
// Parameter order: (disks, source, destination, auxiliary)
```

1. IF $n = 1$ THEN
2. PRINT "Move disk 1 from A to C"
3. RETURN
4. END IF

5. Hanoi($n - 1$, A, B, C)
 // Step 1: move top $n-1$ disks from A
 // onto the middle peg B (C is the helper now)

6. PRINT "Move disk n from A to C"
 // Step 2: the largest disk goes straight home

7. Hanoi($n - 1$, B, C, A)
 // Step 3: move the $n-1$ disks from B
 // onto C (A is the helper now)

```
END ALGORITHM
```

Steps 1 and 3 are the SAME problem, one disk smaller — that is the recursion.

Trace for $n = 3$: the 7 moves appear by themselves

#	Move printed	Where it came from
1	Move disk 1: A $\rightarrow$ C	from hanoi(1, A, C, B)
2	Move disk 2: A $\rightarrow$ B	from hanoi(2, A, B, C)
3	Move disk 1: C $\rightarrow$ B	small disk rejoins on B
4	Move disk 3: A $\rightarrow$ C	the big move — middle of hanoi(3)
5	Move disk 1: B $\rightarrow$ A	unstack the pair again
6	Move disk 2: B $\rightarrow$ C	from hanoi(2, B, C, A)
7	Move disk 1: A $\rightarrow$ C	done — tower rebuilt on C

Exactly $2^3 - 1 = 7$ moves — and this is the program's real output, not a hand-made list.

Tail recursion: when the call is the last act

```
/* normal: work AFTER the call */
int factorial(int n)
{
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

```
/* tail: the call IS the answer */
int factorialTail(int n, int result)
{
    if (n == 0) return result;
    return factorialTail(n - 1,
                        n * result);
}
```

- Normal version: after factorial($n-1$) returns, we still multiply by n — so the frame must wait on the stack.
- Tail version: the answer is carried along in result; when the call returns, there is nothing left to do.
- A frame with nothing left to do need not be kept — compilers can reuse it, making the recursion run like a loop.

One idea ran through the whole unit

Lect 8 Stack ADT on an array

Lect 9 Same ADT on linked nodes + infix $\rightarrow$ postfix

Lect 10 Stacks at work: evaluation + the call stack

LIFO is not a toy: it evaluates every expression your compiler sees and runs every function call your program makes.

Unit I complete

- Postfix evaluation: numbers push, operators pop two and push one; pop order is right first, then left.
- Recursion = base case + recursive case, executed on the call stack: every call pushes a frame, every return pops one.
- Tower of Hanoi: move $n - 1$ away, move the largest, move $n - 1$ back — $2^n - 1$ moves emerge on their own.
- Tail recursion carries the answer downward, so the frame has nothing left to do when the call returns.

EXIT TICKET (a) Evaluate $5\ 1\ 2 + 4 * + 3 -$ showing the stack. (b) How many moves for 4 Hanoi disks, and what is move #8?